Autonomous Software Agents

Deliveroo.js project

Two autonomous agents with BDI control, PDDL crate planning and LLM mission handling

Daniele Buondonno - 267888
Sasha Petkovic - 264689

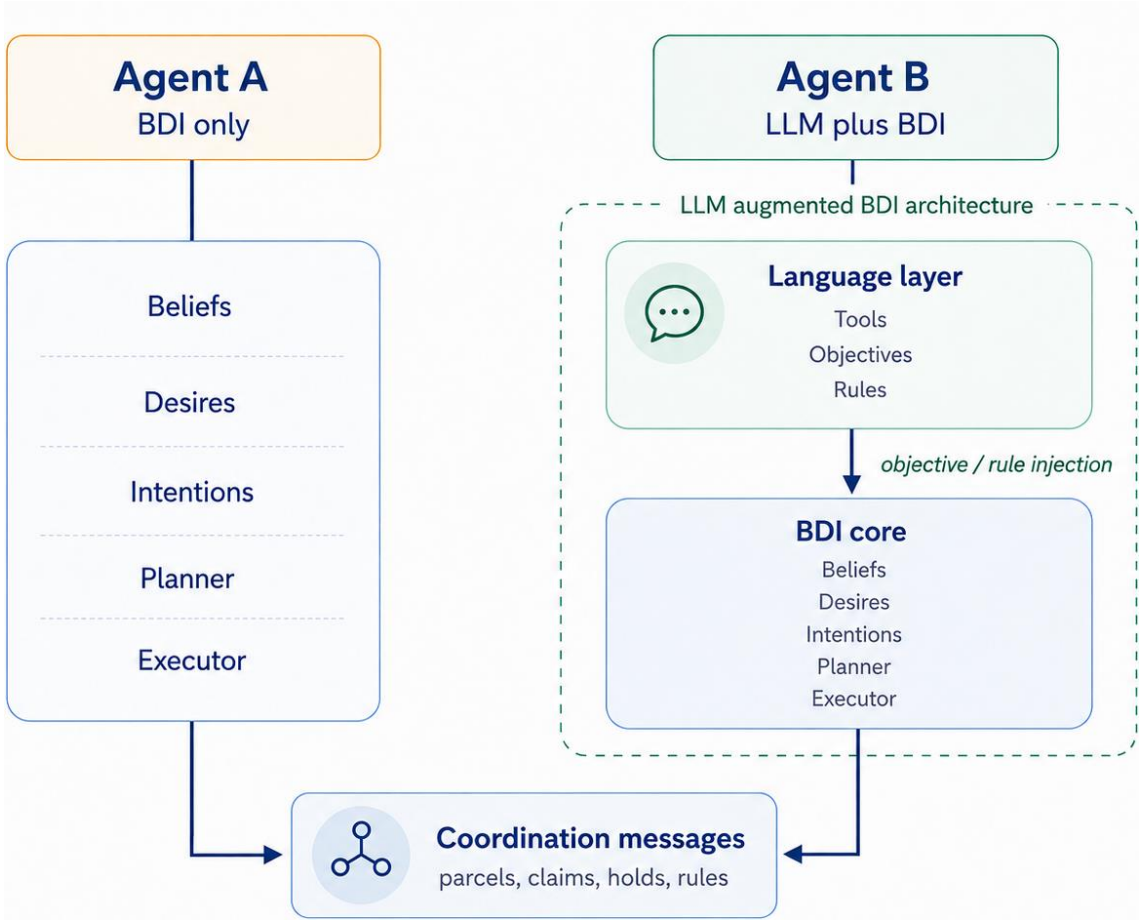
University of Trento
Artificial Intelligence Systems

Roadmap

- System architecture: two agents sharing the same BDI design, with an LLM augmentation layer on Agent B.
- BDI core: belief revision, utility based desire generation and intention revision.
- Planning: BFS for ordinary navigation, dynamic obstacles and PDDL for crate manipulation.
- LLM layer: mission execution, tools, external objectives and persistent strategies.
- Coordination: parcel allocation, coordinated meeting tasks and parcel handoff.

Architecture: two agents, same BDI design

- Agent A is a standard BDI agent. It autonomously collects, delivers, explores and handles crates.
- Agent B has the same BDI core, plus an LLM layer that interprets missions in natural language.
- The LLM layer is not a second physical controller. It creates BDI objectives, persistent rules or coordination messages.
- Both agents keep independent beliefs and planners. They cooperate by exchanging selected information.
- The final team consists of Agent A, a BDI agent, and Agent B, an LLM agent built on the same BDI core.



Physical actions are always performed by the BDI executor.

Beliefs: evidence based state

- The belief set stores what the agent can currently justify, not guaranteed ground truth.
- Parcels are visible, known or carried. A known parcel is removed only when its last tile is observed without it.
- Crates use the same evidence based update, plus a position index for pathfinding and PDDL.
- Fractional coordinates of other agents are retained because they indicate movement between two adjacent tiles and allow both positions to be treated as temporarily occupied.
- Beliefs are updated immediately after confirmed physical actions, such as a successful move, pickup, putdown or crate push; later sensing can still correct the local state if needed.
- Parcels reported by the teammate are added to known parcels, but never replace direct observations.

Key distinction: not observing a parcel is different from observing that its tile is empty.

Desires: utility-based option generation

- The agent considers three main desires: picking up parcel batches, delivering carried parcels, and exploring parcel spawners.
- Distances are shortest path distances, not Manhattan distance.
- Pickup utility evaluates the complete route: agent to batch, then batch to delivery.
- Carried parcels are included because they also decay while the agent makes a detour.
- The selected delivery is the one with best utility, not necessarily the closest tile.

$$U_{\text{pickup}}(b, d) = \frac{[R(b, D) + R(C, D)]m_{\text{stack}}(|C| + |b|)m_{\text{delivery}}(d)}{\max(1, D)}$$

$$R(S, D) = \sum_{p \in S} \max\left(0,\ r_p - \delta D\right)\ m_{\text{parcel}}\left(r_p\right)$$

b: pickup batch, ***C***: carried parcels, ***d***: delivery tile, ***S***: parcel set, ***D***: complete route length, δ: reward decay per move, r_p: current reward of parcel *p*. *Strategy multipliers are 1 when inactive.*

Intention revision: stable but reactive

- Desires are regenerated every BDI cycle, but the current intention is not replaced after every utility change.
- The current intention is matched against the newly generated desires using a stable identifier based on its goal.
- If the intention is no longer valid, the agent selects a new one.
- Otherwise, replacement is controlled by explicit revision rules, not by pure greedy recomputation.
- This prevents oscillation while still reacting to invalid goals and clearly better opportunities.

Mental model

1. Is the current intention still valid?

2. Does a higher-priority objective apply?

3. Is switching worth breaking commitment?

Why it matters

Avoids oscillation without preventing the agent from reacting to meaningful changes.

Intention revision: the actual replacement rules

- Valid external objectives have priority while active, because they correspond to missions accepted from the LLM layer.
- When a delivery requires an active crate task, the agent keeps that intention until the task is completed or the crate configuration changes.
- A valid visible pickup on the current tile is taken immediately, since collecting it requires no additional movement.
- During exploration, the agent remains committed to the selected spawner while that desire is valid, avoiding unnecessary switches between spawners.
- For BDI-generated desires, a new desire must have strictly greater utility to replace the current one.
- A delivery is protected: it can be interrupted only by a pickup with greater utility and shorter distance.

$$\text{delivery replacement rule: } U_{\text{pickup}} > U_{\text{delivery}} \ \wedge \ d_{\text{pickup}} < d_{\text{delivery}}$$

Intention revision combines utility comparison with explicit commitment rules.

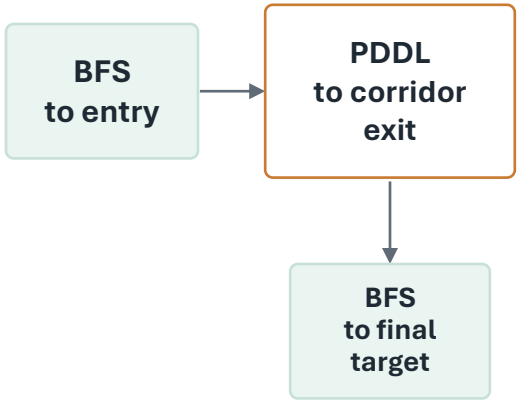
Planner: BFS by default

- BFS is the default planner for ordinary navigation because all movements have uniform cost and it provides exact shortest paths on the grid.
- BFS uses the actual traversability constraints of the map: walls, directional tiles and avoided tiles, while known crates are treated as blocked.
- Planning produces only the next action toward the current intention, rather than committing the executor to an entire path.
- After each successful action, beliefs can be updated and the next BDI cycle can reconsider the current intention before planning again.
- Other agents are treated as temporary obstacles: the planner first searches for a detour, waits if none is available, and eventually defers the intention if the blockage persists.

Reactive planning: execute one action, update the state, then plan again.

PDDL: only for crate manipulation

- PDDL is used only when ordinary BFS fails with known crates treated as blocked.
- The online solver has overhead, so PDDL is avoided for ordinary navigation.
- The planner first detects a possible crate corridor, identifying an entry and an exit around the blocked section.
- The route is optimized as BFS to the entry, PDDL through the crate section, then BFS to the final target.
- Using a closer intermediate goal usually reduces the PDDL plan depth and the online solver work.
- If the local solution fails, global PDDL is used as fallback; returned plans and next actions are validated before execution.



Key idea: use symbolic planning only where changing the environment is actually required.

LLM mission execution: one tool per turn

- Before each model call, a compact context is rebuilt from the mission goal, current BDI beliefs, active strategies and recent tool results.
- At each turn, the model requests one tool or produces a final answer; the tool result is added to memory before the next turn.
- Before execution, the model output is validated so that only one valid tool action is accepted per turn.
- Physical tools such as *go_to*, *pick_up* and *put_down* create external BDI objectives and wait for the BDI layer to report success or failure.
- Tool failures become corrective feedback, allowing the next model turn to choose a different valid approach.
- The execution loop is bounded to 10 model turns, preventing an unresolved mission from running indefinitely.

The LLM decides what should be achieved; the BDI layer plans and executes the physical actions.

Persistent strategies and coordination

- Persistent strategies are interpreted once by the LLM and stored as deterministic BDI rules. They remain active until they are replaced or cleared.
- The supported rules modify stack size, delivery preference, parcel value and avoided tiles.
- Validated strategies are shared with the teammate, so both agents can adapt their behavior consistently.
- When both agents target the same parcel, they compare their BFS distances: the closer agent keeps the pickup, while the other gives it up.
- For coordinated meeting tasks, the two agents move to distinct reachable tiles within the requested radius and wait for each other until the task succeeds or times out.
- Handoff dynamically assigns giver and receiver, keeps track of the same parcel during the exchange and prevents the giver from immediately picking it up again.

The LLM interprets high-level requests once; persistent rules and BDI objectives enforce them during execution.

Thank you for your attention